

A1a PACE-ICE Setup

Conversational AI: Submitting Your Part B (BiLSTM) SLURM Job on PACE-ICE

September 2026

This guide walks you through **submitting and running your Part B (BiLSTM) training job on PACE-ICE via SLURM**, the part of A1a that actually needs the cluster. Part A (CRF) needs no GPU and isn't covered here beyond a one-line note in §3. If you just want the fast path, skim §1 and §2 for setup, then read §4 closely. That's the actual walkthrough.

Read before you touch the cluster: **ICE Cluster Resources** (https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042095) and **Slurm on ICE** (https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042096). Part B's GPUs (H100/H200) live in the **AI Makerspace** pool. Off campus, you'll also need the GT VPN client: enter `vpn.gatech.edu`, then log in with your GT username and password.

1. Get In, Get Your Code and Data There

Log in (either works):

```
ssh <your-gt-username>@login-ice.pace.gatech.edu
```

Or use ICE OnDemand's browser dashboard, **Clusters** → **_ICE Shell Access**. Either way you land on a shared **login node**, which is fine for setup and submitting jobs, but not for training.

Get your repo and data onto scratch storage. `~` has a small quota, and `~/scratch` doesn't, so never put your conda env or the ATIS data in `~`:

```
cd ~/scratch
git clone <your-a1a-repo-url> A1a-Slot-Filling-CRF-BiLSTM
cd A1a-Slot-Filling-CRF-BiLSTM
```

The ATIS data/ folder isn't checked into the starter repo. If it isn't already inside your clone, copy it over with `sftp` from your **local** machine, then move it into place on ICE:

```
sftp <your-gt-username>@login-ice.pace.gatech.edu
sftp> put -r data
sftp> exit

mv ~/data ~/scratch/A1a-Slot-Filling-CRF-BiLSTM/data
ls data/atis/train    # sanity check: seq.in, seq.out, label
```

2. Conda Environment

```
module load anaconda3          # re-run this every new login/shell, it doesn't persist
conda env create --file env.yml
conda activate CAI_SO_FUN
python -c "import sklearn_crfsuite, torch, seqeval; print('ok')"
```

env.yml pins everything both parts need: sklearn-crfsuite (Part A), pytorch plus tensorboard (Part B), seqeval (scoring), and matplotlib (Part C's plot).

3. Part A (CRF), Just a Note

CRF training is CPU-only and fast. Run `python train.py --model crf ...` directly on the login node; no SLURM involved. The rest of this guide covers Part B, which is the part that actually needs the scheduler.

4. Part B (BiLSTM): Submitting a SLURM Job, Full Walkthrough

4.1 Why a batch job, not an interactive session

BiLSTM training needs a GPU, and PACE's GPUs are a shared, contended resource. You don't get to just grab one and run code live the way you would on your laptop. Instead, SLURM makes you describe the job you want to run (the resources it needs and the command to run) in a script. You hand that script to `sbatch`, and SLURM queues it, finds a free GPU node whenever one opens up, runs your script there, and writes the output to a log file. You don't stay connected while it runs. Submit it and walk away, then check back with `squeue` (§4.4) or your log file (§4.5).

This matters for A1a specifically because Part C asks for **four BiLSTM runs** (5%, 10%, 25%, and 100% of training data). You'll submit four jobs, let them queue, and come back for the results.

4.2 Anatomy of the job script

The repo already provides `slurm/train_bilstm.sbatch`. You shouldn't need to write your own from scratch, but you should understand what every line does before you submit it:

```
#!/bin/bash
#SBATCH --job-name=a1a-bilstm
#SBATCH --gres=gpu:H100:1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=02:00:00
#SBATCH --output=slurm/logs/%x-%j.out
#SBATCH --error=slurm/logs/%x-%j.err
```

```
module load anaconda3
conda activate CAI_SO_FUN
```

```
FRACTION="${1:?Usage: sbatch slurm/train_bilstm.sbatch <fraction>}"
```

```
python train.py \
    --model bilstm \
    --train-fraction "${FRACTION}" \
    --checkpoint-dir "checkpoints/bilstm_${FRACTION}" \
    --seed 42 \
    --tensorboard-logdir "runs/bilstm_${FRACTION}"
```

Line by line:

Directive	What it means
--job-name=a1a-bilstm	Label shown in squeue; also feeds %x below.
--gres=gpu:H100:1	Request one H100 GPU . Swap H100 for H200 to queue for the other AI Makerspace GPU type instead.
--ntasks=1	One process; this isn't a multi-node/multi-process job.
--cpus-per-task=8	CPU cores for data loading etc., alongside the GPU.
--mem=32G	RAM reserved for the job.
--time=02:00:00	Hard wall clock limit of 2 hours. SLURM kills the job if it runs longer.
--output / --error	Where stdout/stderr get written. %x is the job name, %j is the job ID, so each submission gets its own log file.

Below the #SBATCH block, it's a normal shell script: load the module, activate the env, then call `train.py`, exactly the CLI you'd run by hand, just parameterized on \$1 (the fraction you pass to sbatch) so one script file covers all four Part C runs.

Before your first submission, create the log directory it writes to (SLURM won't create it for you):

```
mkdir -p slurm/logs
```

4.3 Submitting

Run the four Part C fractions as four separate submissions. They're independent and can sit in the queue and run in parallel:

```
sbatch slurm/train_bilstm.sbatch 0.05
sbatch slurm/train_bilstm.sbatch 0.10
sbatch slurm/train_bilstm.sbatch 0.25
sbatch slurm/train_bilstm.sbatch 1.00
```

Each call returns almost instantly with a line like `Submitted batch job 123456`, and that number is your job ID. **This does not mean training started**; it means the job entered the queue. Use §4.4 to check whether it's actually running yet.

4.4 Checking on your job

```
squeue -u <your-gt-username>
```

You'll see a table with a ST (state) column:

- PD: pending, waiting for a free GPU. Normal, especially close to the deadline when the whole course is competing for the same GPU pool.
- R: running.
- Not listed at all: it finished. Check the log to see whether it finished successfully or crashed.

Made a mistake (wrong fraction, wrong script)? Cancel it by job ID:

```
scancel <job-id>
```

4.5 Reading the logs

Once a job starts (or finishes), its output lands in `slurm/logs/`:

```
tail -f slurm/logs/a1a-bilstm-<job-id>.out      # stdout, live if still running
cat slurm/logs/a1a-bilstm-<job-id>.err          # stderr, Python tracebacks land here
```

If a job disappears from `squeue` sooner than expected, the `.err` file is the first place to look. A crash in your `train_loop()` implementation still ends the SLURM job, just with a nonzero exit code instead of a timeout.

4.6 Watching training progress live (optional)

Each run logs to `runs/bilstm_<fraction>/` via `--tensorboard-logdir`. From a login-node shell:

```
tensorboard --logdir runs --port 6006
```

Then tunnel to it from your **local** machine (separate terminal):

```
ssh -L 6006:localhost:6006 <your-gt-username>@login-ice.pace.gatech.edu
```

and open `http://localhost:6006` in your browser.

4.7 When it's done

A finished job leaves a checkpoint at `checkpoints/bilstm_<fraction>/bilstm_model.pt`. Confirm it's there, then move on to evaluation:

```
ls checkpoints/bilstm_1.00/
python evaluate.py --model bilstm --checkpoint-dir checkpoints/bilstm_1.00 --split test
```

5. Common Pitfalls

- **“command not found: conda” on a new login.** You forgot to `module load anaconda3` again; it doesn't persist across sessions.
- **Job stuck in PD for a long time.** The GPU pool is shared and gets contended near the deadline. Don't wait until the last night to run your Part C sweep.
- **`--gres=gpu:H100:1` job never starts.** Try H200 instead, or run `sinfo` to see what's actually free right now.

- **Job vanishes from squeue almost immediately.** Check `slurm/logs/*.err`; it likely crashed instead of finishing. A common cause is `RuntimeError: Expected train_loop() to save a checkpoint...`, meaning your `train_loop()` didn't write to `args.checkpoint_dir` in the format `evaluate.py`'s `load_model()` expects. That's a bug in your code, not a PACE issue.
 - **Home-directory quota errors.** Your repo or env ended up in `~` instead of `~/scratch`. Move it, see §1.
-

Appendix: Reference Links

- ICE Cluster Resources: https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042095
- Slurm on ICE: https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042096
- GT VPN client download: https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042139
- Storage on ICE: https://gatech.service-now.com/technology?id=kb_article_view&sysparm_article=KB0042094